



Student Quick Start Guide

RAG Lab for Beginners - Step by Step

Welcome! This guide assumes you're new to Python. We'll walk through everything together.



What You'll Need

1. **A computer** (Windows, Mac, or Linux)
 2. **Python installed** (version 3.8 or higher)
 3. **A text editor** (Cursor for AI assisted coding)
 4. **Google Gemini API key** (free - we'll show you how to get it)
 5. **About 3-4 hours** to complete the lab
-



What Are We Building?

Imagine you have a long document (like a textbook chapter). You want to:

1. Upload it to your computer
2. Ask questions about it
3. Get AI-powered answers

That's what we're building! It's called a **RAG system** (Retrieval-Augmented Generation).

Part 1: Getting Set Up (30 minutes)

Step 1: Check if Python is Installed

Open your terminal/command prompt:

- **Windows:** Press Win + R , type cmd , press Enter
- **Mac:** Press Cmd + Space , type terminal , press Enter
- **Linux:** Press Ctrl + Alt + T

Type this and press Enter:

```
python --version
```

What you should see:

```
Python 3.8.0 (or higher)
```

If you see an error: You need to install Python from

<https://www.python.org/downloads/>

Step 2: Create a Project Folder**In your terminal, type these commands one at a time:**

```
# Go to your Documents folder
cd Documents

# Create a new folder called rag_lab
mkdir rag_lab

# Go into that folder
cd rag_lab
```

Check where you are:

```
pwd
# Should show: /Users/YourName/Documents/rag_lab (or similar)
```

Step 3: Download the Project Files**Ask your teacher for the project files, then:**

1. Download `generate_all_files.py`
2. Save it in your `rag_lab` folder
3. In terminal, make sure you're in the `rag_lab` folder:

```
cd Documents/rag_lab
```

Run the file generator:

```
python generate_all_files.py
```

You should see:

Creating RAG Lab files...

- ✓ Created `config.py`
- ✓ Created `document_processor.py`
- ✓ Created `relevance_scorer.py`
- ✓ Created `guardrails.py`
- ✓ Created `rag_app.py`
- ✓ Created `requirements.txt`
- ✓ Created `sample_documents/ai_history.txt`

✓ All files created successfully!

Check your folder - you should now have 7 new files!

Step 4: Set Up a Virtual Environment

What's a virtual environment? It's like a separate workspace for this project, so we don't mess up other Python projects.

Create it:

```
python -m venv rag_env
```

Wait a minute... this creates a new folder called `rag_env`

Activate it:

On Windows:

```
rag_env\Scripts\activate
```

On Mac/Linux:

```
source  
rag_env/bin/activate
```

You should see (rag_env) appear at the start of your terminal line. This means it worked!

Step 5: Install Required Libraries

What are libraries? They're pre-written code that helps us do complex tasks easily.

Install them all at once:

```
pip install -r requirements.txt
```

This will take 5-10 minutes. You'll see lots of text scrolling by. That's normal!

 **First time running will also download AI models (~600MB). Be patient!**

Note: if you get a "cannot build chroma wheel" error, follow the troubleshooting guide to install C++ dev tools for your environment.

Step 6: Get Your Gemini API Key (if you haven't already)

NOTE: If you are under 18, note the Slack message

Options for a 17-year-old Parent/Guardian Account Setup A parent/guardian who is over 18 can create a Google Cloud account, enable the Gemini API, and generate an API key. That key can then be provided to the student for coding projects (with supervision).

1. Go to <https://makersuite.google.com/app/apikey>
2. Sign in with your Google account
3. Click "Create API Key"
4. Copy the key (it looks like: AIzaSyD...)

Set the API key:

On Windows:

```
set GEMINI_API_KEY=paste_your_key_here
```

On Mac/Linux:

```
export GEMINI_API_KEY=paste_your_key_here
```

 **Important: You'll need to do this every time you open a new terminal!**

Part 2: Understanding the Files (10 minutes)

What Files Do You Have?

Let's look at what each file does:

File	What It Does	Do You Edit It?
config.py	Stores settings (like chunk size)	Maybe (change settings)
document_processor.py	Assignment 1 - Processes documents	YES - Fill in TODOs
relevance_scorer.py	Assignment 2 - Scores results	YES - Fill in TODOs
guardrails.py	Assignment 3 - Safety checks	YES - Fill in TODOs
rag_app.py	Assignment 4 - Main app	YES - Fill in TODOs
requirements.txt	List of libraries needed	NO
sample_documents/ai_history.txt	Test document	NO

Part 3: Your First Assignment (45 minutes)

Assignment 1: Document Processor

Goal: Learn how to clean text, split it into chunks, and create embeddings.

Step 1: Open the File

Using VS Code (recommended):

```
code document_processor.py
```

Or just open it in any text editor

Step 2: Find the First TODO

Scroll down until you see:

```
def clean_text(self, text: str) -> str:
    """
    TODO: Implement text cleaning for embedding hygiene
    ...
    """
    # YOUR CODE HERE
    cleaned = text

    return cleaned.strip()
```

Step 3: Understand What It Needs

Read the comments! They tell you:

1. What the function should do
2. What data it receives
3. What it should return

For `clean_text`, we need to:

- Remove extra spaces
- Remove weird characters

- Make the text clean for the AI

Step 4: Write Your Code

Replace # YOUR CODE HERE with:

```
def clean_text(self, text: str) -> str:
    """
    TODO: Implement text cleaning for embedding hygiene
    """
    # Step 1: Remove extra whitespace
    cleaned = re.sub(r'\s+', ' ', text)

    # Step 2: Remove special characters (keep letters, numbers, basic punct
    cleaned = re.sub(r'^\w\s.,!?:\-\'\\"() ]', '', cleaned)

    # Step 3: Remove leading/trailing spaces
    return cleaned.strip()
```

Let's break this down:

- `re.sub(r'\s+', ' ', text)` - Replace any amount of whitespace with a single space
- `re.sub(r'^\w\s.,!?:\-\'\\"() ]', '', cleaned)` - Remove weird characters
- `.strip()` - Remove spaces at the start/end

Step 5: Test Your Code

In terminal, run:

```
python document_processor.py
```

You should see:

```
Testing text cleaning...
Original length: 345, Cleaned length: 320
```

If you see errors: Read the error message! It usually tells you what's wrong.

Common Errors and How to Fix Them

Error 1: ModuleNotFoundError: No module named 'chromadb'

Problem: You didn't install the libraries

Fix:

```
pip install -r requirements.txt
```

Error 2: NameError: name 're' is not defined

Problem: Missing import at top of file

Fix: Make sure the top of the file has:

```
import re
```

Error 3: IndentationError: expected an indented block

Problem: Python needs proper spacing (indentation)

Fix: Make sure your code lines up with the function definition. Use 4 spaces (or Tab).

Part 4: Assignment Workflow (Repeat for Each)

For Each Assignment File:

1. **Open the file** in your text editor
2. **Find all TODO comments** (use Ctrl+F to search "TODO")
3. **Read the comments** to understand what's needed
4. **Look at the hints** in the comments
5. **Write your code** where it says `# YOUR CODE HERE`
6. **Test your code** by running: `python filename.py`
7. **Fix any errors** by reading error messages
8. **Move to next TODO** in the file

The Order:

1.  Assignment 1: `document_processor.py` (3 TODOs)

2.  Assignment 2: `relevance_scorer.py` (4 TODOs)
 3.  Assignment 3: `guardrails.py` (4 TODOs)
 4.  Assignment 4: `rag_app.py` (5 TODOs)
-

Part 5: Testing As You Go

After Each Function:

Run the test at the bottom of the file:

```
python document_processor.py
```

Look for:

-  Green checkmarks or “Success!” messages
-  Red errors or tracebacks

If Tests Pass:

 Great! Move to the next TODO

If Tests Fail:

1. Read the error message carefully
 2. Check your code for typos
 3. Make sure indentation (spacing) is correct
 4. Compare your code to the hints
 5. Ask your teacher or classmate for help
-

Part 6: Running the Final App (After All Assignments)

Once you’ve completed all 4 assignments:

Run the Streamlit app:

```
streamlit run rag_app.py
```

Your browser should open automatically to `http://localhost:8501`

You should see:

- A sidebar to upload documents
- A text box to enter questions
- A search button

Try it out:

1. Upload `sample_documents/ai_history.txt`
2. Ask: "What is the Turing Test?"
3. Click Search
4. See the AI-powered answer!

Getting Help

If You're Stuck:

1. **Read the error message** - It usually tells you what's wrong
2. **Check your indentation** - Python is picky about spaces
3. **Look at the hints** in the code comments
4. **Compare to examples** in `ASSIGNMENT_GUIDE.md`
5. **Ask a classmate** - Explaining helps you both learn
6. **Ask your teacher** - That's what they're there for!

Good Questions to Ask:

- "I'm getting this error: [paste error]. What does it mean?"
- "My function should do X, but it's doing Y. Can you help me debug?"
- "I don't understand what this TODO is asking me to do."

Bad Questions to Ask:

- “Can you just give me the answer?”
 - “It doesn’t work.” (Be specific!)
-

Python Basics You’ll Need

Variables

```
# Storing data
name = "Alice"
age = 17
is_student = True
```

Lists

```
# A collection of items
fruits = ["apple", "banana", "orange"]
first_fruit = fruits[0] # "apple"
```

Dictionaries

```
# Key-value pairs
person = {
    'name': 'Alice',
    'age': 17,
    'grade': 'A'
}
print(person['name']) # "Alice"
```

Functions

```
# Reusable code blocks
def greet(name):
    return f"Hello, {name}!"

message = greet("Alice") # "Hello, Alice!"
```

Loops

```
# Do something multiple times
for i in range(5):
    print(i) # Prints 0, 1, 2, 3, 4

# Loop through a list
for fruit in fruits:
    print(fruit)
```

If Statements

```
# Make decisions
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Success Checklist

Before you submit, make sure:

- ☐ All 4 assignment files run without errors
 - ☐ You've filled in all TODO sections
 - ☐ Test outputs look correct
 - ☐ The Streamlit app launches successfully
 - ☐ You can upload a document and get answers
 - ☐ Your code has comments explaining what it does
 - ☐ You've saved all your files
-

Tips for Success

1. **Start early** - Don't wait until the night before!
2. **Test frequently** - Run your code after every change

3. **Read error messages** - They're trying to help you
 4. **Comment your code** - Future you will thank you
 5. **Take breaks** - Your brain needs rest
 6. **Help others** - Teaching helps you learn
 7. **Have fun** - You're building something cool!
-

What You're Learning

This project teaches you:

-  Python programming basics
-  Working with AI and machine learning
-  Vector databases and embeddings
-  Text processing and natural language
-  Building web applications
-  Software engineering practices
-  Debugging and problem-solving

These are real skills used by professional AI engineers!

Quick Reference

Terminal Commands You'll Use:

```
cd Documents/rag_lab          # Go to project folder
source rag_env/bin/activate    # Activate environment (Mac/Linux)
rag_env\Scripts\activate      # Activate environment (Windows)
python document_processor.py   # Test a file
streamlit run rag_app.py      # Run the app
```

VS Code Shortcuts:

- `Ctrl + /` - Comment/uncomment line

- Ctrl + F - Find (search for "TODO")
 - Ctrl + S - Save file
 - Ctrl + Z - Undo
-

Ready to start? Go to Assignment 1: document_processor.py and find the first TODO!

Good luck! 🚀